Corso di Programmazione Web
Simulazione di esame – 28 Maggio 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Fornire esempi di richieste e risposte per mostrare il funzionamento.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.

Progettare un sistema web che gestisca le prenotazioni in un ristorante. Il sistema deve prevedere:

- Un'API per registrare un nuovo ristorante (nome, indirizzo, numero massimo di coperti);
- Un'API per ottenere la lista di tutti i ristoranti disponibili;
- Un'API per visualizzare le informazioni di un ristorante dato il suo ID;
- Un'API per prenotare un tavolo indicando nome, numero di persone e orario.

Domanda 1: Definizione delle API (15 punti)

- Definire i modelli di dati per richiesta e risposta. Inserire I tipi di dato per ogni campo ed eventuali constraint – 3 punti in totale
- Definire endpoint API (URL + metodo HTTP) e descriverlo testualmente dove necessario – 2 punti per ciascuna API
- Specificare i parametri di richiesta e le risposte previste, fornendo un esempio di richiesta con successo per ognuna– 1 punto per ciascuna API

Domanda 2: Gestione degli Errori (5 punti)

- Descrivere i codici di errore rilevanti (400, 404, 500) – 1 punto
- Fornire almeno due esempi concreti di risposta di errore – 2 punti ciascuno (max 4 punti in totale)

Alcune domande possibili da 3 punti:

1. Supponiamo che il sistema venga usato contemporaneamente da migliaia di utenti durante le ore di punta (es. cena del sabato). Indica almeno **due strategie tecniche** che potresti applicare per migliorare la **scalabilità e le performance** del sistema. Spiega in breve il ruolo di ciascuna.

2. Descrivi **due buone pratiche per il deployment automatico** del sistema di prenotazioni ristoranti, in modo che possa essere facilmente aggiornato e mantenga l'affidabilità in produzione.
3. Il sistema attuale permette a chiunque di fare prenotazioni o registrare ristoranti. Spiega **come introdurre autenticazione** per gli utenti e **come gestire ruoli diversi** (es. utente normale vs. gestore ristorante).
4. Dopo che un utente effettua una prenotazione, il sistema deve inviargli un'email di conferma. Spiega **perché è meglio gestire l'invio in modo asincrono**, e **quali strumenti potresti usare per implementarlo**.
5. Per garantire che le API funzionino correttamente, è importante testarle. Descrivi almeno **due tipi di test automatici** che scriveresti per questo progetto e **quale parte del sistema andrebbero a verificare**.

MODELLO RISTORANTE ⟶ {

"id" : int : 1 , // ASSEGNATO DAL SERVER, OBBLIGATORIO IN RISPOSTA, UNIVOCO

"NAME" : STR : "TRATTORIA BELLA NAPOLI", // OBBLIGATORIO, NON VUOTO

"ADDRESS" : STR : "VIA ROMA 10, NAPOLI", // OBBLIGATORIO, NON VUOTO

"MAXGUESTS" : INT : 50 , // OBBLIGATORIO , MIN = 1 , MAX = 200

}

MODELLO PRENOTAZIONE ⟶ {

"idReservation" : INT : 10 , // ASSEGNATO DAL SERVER, OBBLIGATORIO IN RISPOSTA

"RESTAURANT" : INT : 1 , // id DEL RISTORANTE, PRESO DAL PATH

"NAME" : STR : "LUCA BIANCHI", // OBBLIGATORIO, NON VUOTO

"PEOPLE" : INT : 4 , // OBBLIGATORIO , MIN = 1 , MAX = 10

"TIME" : DATETIME | STR : "2026-05-30T20:00", // OBBLIGATORIO, FORMATO DATA-ORARIO VALIDO

"STATUS" : STR : "CONFIRMED" // ASSEGNATO DAL SERVER, VALORI AMMESSI: CONFIRMED

}

API 1 - REGISTRAZIONE NUOVO RISTORANTE

POST /RESTAURANTS ⟶

SI USA POST PERCHÈ L'UTENTE INVIA DATI AL SERVER PER CREARE UNA NUOVA RISORSA RISTORANTE

⟵ 201 CREATED

{

"NAME" : STR : "TRATTORIA BELLA NAPOLI", // OBBLIGATORIO, NON VUOTO

"ADDRESS" : STR : "VIA ROMA 10, NAPOLI", // OBBLIGATORIO, NON VUOTO

"MAXGUESTS" : INT : 50 // OBBLIGATORIO , MIN = 1 , MAX = 200

}

⟶

{

"id" : 1,

"MESSAGE" : "RISTORANTE REGISTRATO CON SUCCESSO"

}

OPPURE, IN FORMA PIÙ COMPLETA

{

"RESTAURANT" : {

"NAME" : STR : "TRATTORIA BELLA NAPOLI",

"ADDRESS" : STR : "VIA ROMA 10, NAPOLI",

"MAXGUESTS" : INT : 50 // OBBLIGATORIO , MIN = 1 , MAX = 200

},

"MESSAGE" : "RISTORANTE REGISTRATO CON SUCCESSO"

}

API 2 - LISTA DEI RISTORANTI DISPONIBILI

GET /RESTAURANTS ⟶

SI USA GET PERCHÈ L'UTENTE RICHIEDE UNA LISTA DI RISORSE SENZA MODIFICARE DATI SUL SERVER. id È UN PATH PARAMETER PERCHÈ IDENTIFICA DIRETTAMENTE LA RISORSA RISTORANTE RICHIESTA

⟵ 200 OK

NESSUN BODY DEL MODELLO DI RICHIESTA

{

"id" : 1 ,

"NAME" : "TRATTORIA BELLA NAPOLI",

"ADDRESS" : "VIA ROMA 10, NAPOLI",

"MAXGUESTS" : 50

}

API 3 - DETTAGLIO DI UN RISTORANTE DATO IL SUO ID

GET /RESTAURANTS/{id} ⟶

NESSUN BODY DEL MODELLO DI RICHIESTA

⟵ 200 OK

{

"id" : 1 ,

"NAME" : "TRATTORIA BELLA NAPOLI",

"ADDRESS" : "VIA ROMA 10, NAPOLI",

"MAXGUESTS" : 50

}

API 4 - PRENOTAZIONE TAVOLO

POST /RESTAURANTS/{id}/RESERVATIONS →

SI USA POST PERCHÈ L'UTENTE INVIA DATI PER CREARE UNA NUOVA PRENOTAZIONE. id È UN PATH PARAMETER E RAPPRESENTA IL RISTORANTE PRESSO CUI SI VUOLE PRENOTARE

← 201 CREATED

È ACCETTABILE ANCHE 200 OK, MA 201 CREATED È PIÙ PRECISO PERCHÈ VIENE CREATA UNA NUOVA RISORSA PRENOTAZIONE

```
{
  "NAME": STR: "LUCA BIANCHI",   //OBBLIGATORIO, NON VUOTO
  "PEOPLE": INT: 4,   // OBBLIGATORIO, MIN=1, MAX=10
  "TIME": DATETIME | STR: "2026-05-30T20:00"   //OBBLIGATORIO, FORMATO VALIDO
}
```

NON INSERISCO RESTAURANT NEL BODY PERCHÈ IL RISTORANTE È GIÀ IDENTIFICATO NEL PATH

→

```
{
  "IDRESERVATION": 10,
  "RESTAURANT": 1,
  "NAME": "LUCA BIANCHI",
  "PEOPLE": 4,
  "TIME": "2026-05-30T20:00",
  "STATUS": "CONFIRMED",
  "MESSAGE": "PRENOTAZIONE REGISTRATA CON SUCCESSO"
}
```

- **400 BAD REQUEST** → RAPPRESENTA UNA RICHIESTA ERRATA DA PARTE DEL CLIENT: BODY MANCANTE, JSON MALFORMATO, CAMPI OBBLIGATORI MANCANTI O RICHIESTA COSTRUITA MALE. NEL SISTEMA RISTORANTE SI PUÒ USARE QUANDO MANCA NAME, ADDRESS O MAXGUESTS, MANCA NAME, PEOPLE O TIME NELLA PRENOTAZIONE.

POST /RESTAURANTS →

← 400 BAD REQUEST

```
{
  "NAME": "TRATTORIA BELLA NAPOLI",
  "MAXGUESTS": 50
}
```

→

```
{
  "ERROR": "CAMPO ADDRESS MANCANTE"
}
```

- **404 NOT FOUND** → RAPPRESENTA UNA RISORSA NON TROVATA. LA RICHIESTA ARRIVA AL SERVER, MA LA RISORSA IDENTIFICATA DALL'URL O DALL'ID NON ESISTE. NEL SISTEMA RISTORANTE SI USA QUANDO L'UTENTE CHIEDE UN RISTORANTE INESISTENTE, L'ENDPOINT RICHIESTO NON ESISTE

GET /RESTAURANTS/42 →

← 404 NOT FOUND

```
{
  "ERROR": "RISTORANTE CON ID 42 NON TROVATO"
}
```

- **422 UNPROCESSABLE ENTITY** → RAPPRESENTA UNA RICHIESTA SINTATTICAMENTE CORRETTA, MA CON DATI CHE VIOLANO VINCOLI DI VALIDAZIONE O REGOLE LOGICHE. NEL SISTEMA RISTORANTE SI PUÒ USARE QUANDO PEOPLE È MINORE DI 1 OPPURE MAGGIORE DI 10, TIME HA UN FORMATO NON VALIDO, L'ORARIO È NEL PASSATO

POST /RESTAURANTS/1/RESERVATIONS →

← 422 UNPROCESSABLE ENTITY

```
{
  "NAME": "LUCA BIANCHI",
  "PEOPLE": -10,
  "TIME": "2026-05-30T20:00"
}
```

→

```
{
  "ERROR": "NUMERO DI PERSONE NON VALIDO"
}
```

- **500 INTERNAL SERVER ERROR** → RAPPRESENTA UN ERRORE INTERNO LATO SERVER. LA RICHIESTA PUÒ ANCHE ESSERE CORRETTA, MA IL SERVER NON RIESCE A COMPLETARLA PER UN PROBLEMA INTERNO. NEL SISTEMA RISTORANTE SI USA QUANDO IL DATABASE NON È RAGGIUNGIBILE, IL SERVER NON RIESCE A SALVARE UN RISTORANTE

POST /RESTAURANTS/1/RESERVATIONS →

← 500 INTERNAL SERVER ERROR

```
{
  "NAME": "LUCA BIANCHI",
  "PEOPLE": 4,
  "TIME": "2026-05-30T20:00"
}
```

→

```
{
  "ERROR": "ERRORE INTERNO DEL SERVER. RIPROVA PIÙ TARDI"
}
```

1. SCALABILITÁ E PERFORMANCE DURANTE LE ORE DI PUNTA ⟶ PER MIGLIORARE LA SCALABILITÀ E LE PERFORMANCE DEL SISTEMA DURANTE MOMENTI DI ALTO CARICO, COME LA CENA DEL SABATO, SI PUÒ PRIMA DI TUTTO SEPARARE LE OPERAZIONI VELOCI DA QUELLE LENTE USANDO UNA CODA DI PROCESSI. IL SERVER RICEVE LA RICHIESTA DELL'UTENTE E METTE IN CODA LE OPERAZIONI PIÙ PESANTI, COME L'INVIO DI EMAIL O DI ALTRE ELABORAZIONI SECONDARIE. UN PROCESSO SEPARATO POI RILEVA I TASK DALLA CODA E LI ESEGUE. IN QUESTO MODO IL SERVER PRINCIPALE NON RIMANE BLOCCATO E PUÒ CONTINUARE A RICEVERE NUOVE RICHIESTE. UNA SECONDA STRATEGIA CONSISTE NELL'AUMENTARE LE RISORSE DISPONIBILI AL SERVER, PER ESEMPIO USANDO UNA MACCHINA CON PIÙ CPU, PIÙ RAM O UN SERVIZIO CLOUD CON PRESTAZIONI SUPERIORI. QUESTA SOLUZIONE PERMETTE DI GESTIRE PIÙ RICHIESTE CONTEMPORANEAMENTE. SE IL CARICO CRESCE ANCORA, SI PUÒ DISTRIBUIRE IL LAVORO SU PIÙ PROCESSI O PIÙ MACCHINE, COSÍ DA NON CONCENTRARE TUTTE LE RICHIESTE SU UN SOLO SERVER.

2. BUONE PRATICHE PER IL DEPLOYMENT AUTOMATICO ⟶ UNA BUONA PRATICA PER IL DEPLOYMENT AUTOMATICO È USARE CONTAINER, PERCHÈ PERMETTONO DI IMPACCHETTARE L'APPLICAZIONE CON LE SUE DIPENDENZE E RENDERE PIÙ SEMPLICE IL PASSAGGIO DALL'AMBIENTE DI SVILUPPO ALL'AMBIENTE DI PRODUZIONE. IN QUESTO MODO L'APPLICAZIONE PUÒ ESSERE AGGIORNATA O REINSTALLATA CON MENO RISCHI DI ERRORE DOVUTI A DIFFERENZE TRA AMBIENTI. UNA SECONDA BUONA PRATICA È USARE WORKFLOW DI CONTINOUS INTEGRATION E CONTINOUS DEPLOYMENT. QUANDO VIENE FATTO UN PUSH SUL REPOSITORY, IL SISTEMA PUÒ ESEGUIRE AUTOMATICAMENTE CONTROLLI, TEST E POI DEPLOYMENT. QUESTO RIDUCE IL RISCHIO DI METTERE IN PRODUZIONE CODICE NON VERIFICATO E RENDE GLI AGGIORNAMENTI PIÙ RAPIDI E CONTROLLATI. CONTINOUS INTEGRATION AUTOMATIZZA LA VERIFICA DEL CODICE, CONTINOUS DEPLOYMENT AUTOMATIZZA IL RILASCIO IN PRODUZIONE.

3. AUTENTICAZIONE E RUOLI DIVERSI ⟶ PER INTRODURRE AUTENTICAZIONE, IL SISTEMA DEVE IDENTIFICARE L'UTENTE PRIMA DI PERMETTERGLI DI COMPIERE CERTE OPERAZIONI. L'UTENTE PUÒ EFFETTUARE LOGIN CON CREDENZIALI, AD ESEMPIO EMAIL E PASSWORD. SE LE CREDENZIALI SONO CORRETTE, IL SERVER CREA UNA SESSIONE O UN TOKEN CHE VERRÁ USATO NELLE RICHIESTE SUCCESSIVE PER RICONOSCERE L'UTENTE. DOPO L'AUTENTICAZIONE SERVE L'AUTORIZZAZIONE, CIOÈ IL CONTROLLO DEI PERMESSI. IL SISTEMA PUÒ DISTINGUERE ALMENO DUE RUOLI: UTENTE NORMALE E GESTORE RISTORANTE. L'UTENTE NORMALE PUÒ VISUALIZZARE I RISTORANTI E CREARE PRENOTAZIONI PER SÈ STESSO. IL GESTORE RISTORANTE PUÒ REGISTRARE UN RISTORANTE, MODIFICARE I DATI DEL PROPRIO RISTORANTE E VISUALIZZARE LE PRENOTAZIONI RELATIVE AL PROPRIO RISTORANTE. UN EVENTUALE AMMINISTRATORE PUÒ AVERE PERMESSI PIÙ AMPI SUL SISTEMA. SE UN UTENTE AUTENTICATO PROVA A FARE UN'OPERAZIONE PER CUI NON HA PERMESSO, IL SERVER DEVE RIFIUTARE LA RICHIESTA, AD ESEMPIO CON 403 FORBIDDEN.

1. Invio email di conferma in modo asincrono
   È meglio inviare l'email di conferma in modo asincrono perché l'invio di una mail può essere lento o può dipendere da un servizio esterno momentaneamente non disponibile. Se il server aspettasse il completamento dell'invio prima di rispondere all'utente, la richiesta di prenotazione resterebbe bloccata più a lungo. Questo peggiorerebbe l'esperienza utente e ridurrebbe la capacità del server di gestire altre richieste.
   Una soluzione migliore è confermare subito la prenotazione, salvare un task di invio email in una coda e farlo eseguire da un processo separato. In questo modo il server web resta disponibile per altre richieste, mentre l'invio della mail viene gestito in background. Il materiale sui processi asincroni indica proprio l'invio di email come esempio di elaborazione in background adatta alla gestione asincrona

1. Test automatici per verificare le API
   Un primo tipo di test automatico riguarda le richieste corrette. Per esempio, si può testare che una POST /restaurants con nome, indirizzo e numero massimo di coperti validi restituisca lo status code corretto e l'ID del nuovo ristorante. Allo stesso modo, si può testare che GET /restaurants restituisca una lista di ristoranti e che GET /restaurants/{id} restituisca il ristorante corretto.
   Un secondo tipo di test riguarda i casi di errore. Per esempio, si può inviare una prenotazione con people uguale a -10 oppure senza il campo name e verificare che il server risponda con lo status code previsto, come 400 o 422, senza bloccarsi. Si può anche testare la richiesta di un ristorante inesistente, verificando che il server restituisca 404 Not Found. Il materiale sul testing delle web app evidenzia che non basta controllare che il codice "giri", ma bisogna verificare risposta HTTP, status code, contenuto JSON, input malformati, risorse inesistenti ed errori interni.